

## 2.35. Le petit rond : $8 \times 8$

Le même rond, dans une tuile de  $8 \times 8$  : Tuile ROND\_8, posée avec `sprite()`. Quatre fois moins de pixels.

### 1. Ce qu'il faut comprendre

**La console ne sait pas agrandir ni réduire un dessin.** Chaque pixel du dessin donne exactement un pixel à l'écran. Pour un rond plus petit, il faut **un autre dessin**, avec moins de pixels. C'est ce que montrent les leçons 2.35 à 2.39.

**Ici, le rond fait  $8 \times 8$  pixels** : c'est une **tuile**, la plus petite taille. Il s'écrit `Tuile ROND_8 = {...}` avec 8 rangées de 8 signes, dans le fichier `tuile_ROND_8.cpp`. Le `_8` dans le nom dit sa taille.

**Il se pose avec `sprite()`**, qui place un seul lutin de  $8 \times 8$  au pixel près : `sprite(0, 76, 60, ROND_8)`.  $76 = (160 - 8) / 2$  : le milieu de l'écran. Il faut `#include <sprite>` et `#include <Tuile>`.

**Compare avec le 2.01** : le rond de  $16 \times 16$  avait une vraie courbe. En  $8 \times 8$ , il n'a plus que deux marches de chaque côté : on le reconnaît, mais c'est presque un octogone.

**Dans la galerie « Les modèles »**, le bouton  $8 \times 8$  en haut donne toutes les formes à cette taille : ROND\_8, CARRE\_8, et les autres.

**Essaie** : dans `tuile_ROND_8.cpp`, change un `-` en `#` et regarde le pixel changer sur l'écran : en  $8 \times 8$ , chaque pixel compte.

#### Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <sprite>` → [0.69.4. La fonction `sprite\(\)` — un lutin au pixel près](#)
- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)

### 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **455 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

|                    |       |                                                                       |
|--------------------|-------|-----------------------------------------------------------------------|
| démarrage          | 57 o  | ← la console, à l'allumage — la mise en route de la console           |
| main()             | 44 o  | ← démarrage — ta fonction                                             |
| VBlank             | 8 o   | ← la console, à chaque image — compte les images, 60 fois par seconde |
| AttendreImage      | 28 o  | ← <code>image()</code> — attend l'image suivante                      |
| AttendreVBlank     | 16 o  | ← démarrage — attend que l'écran soit libre                           |
| AttendreAcces      | 12 o  | ← <code>EcrireTexte</code> — attend de pouvoir écrire une case        |
| EcrireTexte        | 10 o  | ← <code>texte()</code> — écrit un texte, lettre par lettre            |
| RangerLutins       | 11 o  | ← démarrage — range les 40 lutins hors de l'écran                     |
| InstallerTransfert | 15 o  | ← démarrage — installe la copie des lutins en mémoire rapide          |
| CopierTuiles       | 76 o  | ← démarrage — copie les lettres et les dessins en mémoire vidéo       |
| EffacerCarte       | 14 o  | ← démarrage — vide les deux cartes de l'écran                         |
| Tuiles             | 144 o | ← <code>CopierTuiles</code> — 8 lettres sur 43, et 1 tuile dessinée   |

```

DonneesTransfert      10 o ← InstallerTransfert  – la petite routine qui copie les lutins
"PETIT ROND"          10 o ← texte() – un texte gravé
Lettres gravées : DEINOPRT
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes
Fonctions du langage appelées :
  sprite() × 1      23 o sur place, dans main()
  texte() × 1       11 o sur place, dans main() + EcrireTexte (10 o)
  image() × 1        6 o sur place, dans main() + AttendreImage (28 o)

```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// ---- 2.35 Le petit rond : 8 × 8 ----
// Le rond dans une tuile de 8 × 8, au milieu de l'écran.

#include <texte>           // texte() : écrire un mot
#include <sprite>          // sprite() : poser un lutin de 8 × 8
#include <Tuile>           // Tuile : un dessin de 8 × 8
#include "tuile_ROND_8.cpp" // le petit rond, dans son fichier

```

## 2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

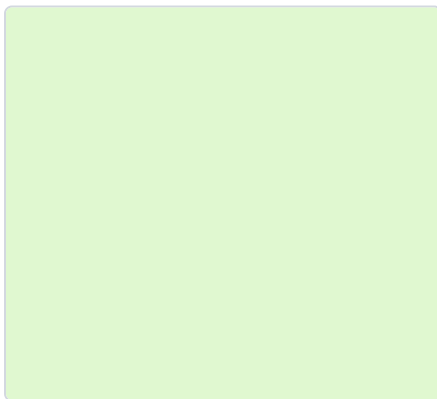
int main() {
  sprite(0, 76, 60, ROND_8); // le lutin 0, en (76, 60)
  texte(5, 10, "PETIT ROND"); // le nom, sous la forme

  while (true) {
    image();
  }
}

```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce qu'on doit voir

Un petit rond de 8 × 8 au milieu de l'écran, et PETIT ROND dessous.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le petit rond est posé en (76, 60) (76, 60)

✓ PETIT ROND est écrit dessous

---

Ensuite : **2.36. Le grand rond : 32 × 32** — série 2 : Les formes géométriques. · gameboy3 — écrire une cartouche Game Boy en C++